AHB5 SUBORDINATE

Design & Verification Report

Prepared by
Omar Ahmed Abdelaty
Digital IC Design & Verification Engineer

SystemVerilog / UVM Methodology

2026

omarahmedabelaty231@gmail.com

1. Executive Summary

This report documents the complete design and verification of an AHB5-compliant subordinate (slave) memory controller implemented in SystemVerilog. The design fully conforms to the AMBA AHB5 protocol specification and has been validated using two complementary verification methodologies: a directed self-checking testbench and a structured Universal Verification Methodology (UVM) environment.

The subordinate successfully implements a 1 KB word-addressable memory array with support for byte-lane write strobes (HWSTRB), address-phase error detection with a two-cycle ERROR response, and combinatorial read data output. All verification scenarios passed with zero errors, confirming functional correctness of the design.

2. Introduction & Background

2.1 AMBA AHB5 Protocol Overview

The Advanced Microcontroller Bus Architecture (AMBA) High-performance Bus version 5 (AHB5) is an ARM-defined on-chip interconnect standard widely adopted in embedded SoC designs. AHB5 introduces byte-lane write strobes (HWSTRB) over its predecessors, enabling fine-grained byte-enable control during write transfers.

The protocol separates each transfer into two distinct phases handled on successive clock edges:

- Address Phase: HSEL, HADDR, HTRANS, HWRITE, HSIZE, and HWSTRB are sampled when HREADY is HIGH.
- Data Phase: HWDATA is valid for writes; HRDATA is driven by the subordinate for reads.

2.2 Project Objectives

- Design a fully AHB5-compliant subordinate memory controller in synthesizable SystemVerilog.
- Implement a two-cycle ERROR response for misaligned address transfers.
- Validate functional correctness using a directed self-checking testbench with a reference memory model.
- Build a reusable UVM verification environment with constrained-random stimulus and automated scoreboard checking.

3. DUT Architecture

3.1 Module Interface

The top-level module ahb5_subordinate is parameterized across three dimensions, enabling reuse across different bus widths and memory configurations:

- ADDR_WIDTH (default: 32): Width of the address bus in bits.
- DATA_WIDTH (default: 32): Width of the data bus in bits.

- MEM_DEPTH (default: 1024): Number of 32-bit words in the internal memory array.

3.2 Port Definitions

Signal	Direction	Width	Description
HCLK	Input	1-bit	System clock — all transfers synchronized on rising edge
HRESETn	Input	1-bit	Active-low asynchronous reset; clears memory and FSM
HSEL	Input	1-bit	Subordinate select; qualifies transfers for this slave
HREADY	Input	1-bit	Global bus-ready; transfer proceeds when HIGH
HADDR	Input	32-bit	Transfer address (byte-addressed, word-aligned enforced)
HTRANS	Input	2-bit	Transfer type: IDLE (00), BUSY (01), NONSEQ (10), SEQ (11)
HWRITE	Input	1-bit	Direction: HIGH = write, LOW = read
HSIZE	Input	3-bit	Transfer size: 000=byte, 001=half-word, 010=word (32-bit)
HWDATA	Input	32-bit	Write data bus (sampled one cycle after address phase)
HWSTRB	Input	4-bit	AHB5 byte-enable strobes; each bit enables one byte lane
HRDATA	Output	32-bit	Read data bus driven combinatorially from memory array
HREADYOUT	Output	1-bit	Slave ready signal; LOW during error response second cycle
HRESP	Output	1-bit	Response: LOW = OKAY, HIGH = ERROR (misaligned address)

3.3 Finite State Machine

The subordinate employs a two-state Moore FSM to manage normal and error transfer flows:

ST_IDLE (Normal Operation State)

- Samples address-phase signals when valid_transfer is asserted (HSEL & HREADY & HTRANS[1]).
- Checks address alignment using the is_aligned() function for HSIZE values 001 (half-word) and 010 (word).
- For aligned transfers: latches address, direction, and size into the data_phase register; asserts HREADYOUT.

- For misaligned transfers: asserts HRESP=1, deasserts HREADYOUT, transitions to ST_ERROR_2.
- Executes write data to memory or drives HRDATA combinatorially for reads when data_phase.valid is set.

ST_ERROR_2 (Error Response Second Cycle)

- Maintains HRESP=1 for the mandatory second error-response cycle per the AHB5 specification.
- Reasserts HREADYOUT=1 to release the bus and unconditionally returns to ST_IDLE.

3.4 Key Design Decisions

Combinatorial HRDATA (Production Version)

The production DUT (ahb5_subordinate.sv) drives HRDATA combinatorially via an always_comb block. This eliminates a one-cycle read latency present in the earlier registered version (ahb.sv), achieving zero-wait-state reads. The combinatorial path is: HRDATA = mem[data_phase.addr[11:2]] when data_phase.valid is asserted and the transfer is a read.

AHB5 Byte Strobes

The HWSTRB input enables byte-granular writes. During the data phase, each of the four byte lanes is written independently:

```
if (HWSTRB[i]) mem[addr[11:2]][(i*8)+:8] <= HWDATA[(i*8)+:8];
```

This allows the initiator to update any combination of bytes within a 32-bit word without disturbing the remaining bytes.

Word-Addressed Memory Indexing

The physical memory array is indexed by addr[11:2], discarding the two least-significant address bits. This naturally enforces 4-byte (word) alignment of the memory word selected, while HWSTRB handles sub-word byte selection.

4. Verification Strategy

A two-pronged verification strategy was applied: a directed self-checking testbench for rapid targeted testing, followed by a UVM constrained-random environment for broader functional coverage.

4.1 Directed Self-Checking Testbench (ahb_tb.sv)

This testbench instantiates the DUT directly and drives AHB5 transfers via task-based stimulus. A reference memory (ref_mem) mirrors all writes using the same byte-strobe logic; read responses are compared in real time.

Reference Model Pipeline

The testbench maintains a two-stage pipeline to correctly model the AHB5 address-to-data phase latency:

- addr_latch: Captures the current address-phase signals (HSEL, HTRANS, HADDR, HWRITE) each clock cycle when HREADYOUT is asserted.
- data_latch: Advances the addr_latch content one cycle later, aligning captured address information with the arriving data phase.
- On each valid data phase, if the operation was a write, ref_mem is updated with HWSTRB-masked bytes. If a read, HRDATA is compared against ref_mem.

4.2 UVM Environment Architecture

The UVM environment is structured around the standard layered testbench architecture. All components are compiled together via ahb_uvm_pkg.sv and connected through the virtual interface mechanism.

File	Type	Purpose
ahb5_subordinate.sv	RTL / DUT	AHB5 subordinate memory slave — production version
ahb.sv	RTL (alt)	Earlier version of subordinate with registered HRDATA
ahb_if.sv	Interface	SystemVerilog interface bundling all AHB5 signals
ahb_uvm_pkg.sv	UVM Package	Compilation package including all UVM components
top.sv	Top-level TB	Instantiates DUT, interface, clock/reset, starts UVM test
ahb_seq_item.sv	UVM Object	Sequence item with randomization constraints
ahb_random_seq.sv	UVM Sequence	Generates 10 random write-then-read-back pairs
ahb_driver.sv	UVM Component	Drives AHB5 address and data phases onto the interface
ahb_monitor.sv	UVM Component	Captures address & data phases; publishes transactions
ahb_scoreboard.sv	UVM Component	Byte-accurate reference model; flags mismatches
ahb_agent.sv	UVM Component	Bundles sequencer, driver, and monitor; exposes analysis port
ahb_env.sv	UVM Component	Connects agent analysis port to scoreboard
ahb_test.sv	UVM Test	Top-level test class; creates env, starts random sequence
ahb_tb.sv	Self-check TB	Directed self-checking testbench with reference memory model

Sequence Item (ahb_seq_item)

The transaction object carries all AHB5 transfer fields. Randomization is constrained to ensure legal, synthesizable transfers:

- c_addr: Address must be below 0x400 (within the 1 KB memory) and word-aligned (haddr[1:0] == 2'b00).
- c_size: Transfer size fixed to 3'b010 (32-bit word) for all random transactions.

Driver (ahb_driver)

Retrieves sequence items from the sequencer and drives the two-phase AHB5 protocol onto the virtual interface. Address-phase signals (HSEL, HTRANS, HADDR, HWRITE, HSIZE) are driven first; HWDATA and HWSTRB are applied one cycle later in the data phase. A wait loop on vif.hready handles back-pressure.

Monitor (ahb_monitor)

Passively observes the interface. It captures the address phase on assertion of hsel & hready & htrans[1], then completes the transaction object in the following data phase by sampling hwdata/hwstrb (writes) or hrdata (reads). Completed transactions are broadcast to subscribers via the analysis port.

Scoreboard (ahb_scoreboard)

Maintains a byte-granular associative array (ref_mem) indexed by byte address. On receiving a write transaction, it applies byte-strobe masking to update the model. On a read transaction, it reconstructs the expected 32-bit word from the four individual byte entries and compares against the actual hrdata. Mismatches trigger UVM_ERROR messages.

Random Sequence (ahb_random_seq)

Generates 10 paired transactions per run. Each pair consists of a fully randomized write followed by a read to the same address, creating a write-then-readback pattern that provides automatic correctness checking for every random address and data combination exercised.

5. Test Scenarios & Results

Test Scenario	Stimulus	Expected Result
Basic Write & Read	Write 0xDEADBEEF to 0x0000_0100, read back	HRDATA matches written data; SCB PASS
Byte-Lane Strobe	Write with HWSTRB=4'b0001 (byte 0 only)	Only byte 0 updated; others unchanged in ref_mem
Sequential W/R Burst	Write 0x1111_1111 and 0x2222_2222, read both	Both reads return correct data in order
Randomized W-then-R	10 random write/read pairs via UVM sequence	All scoreboard checks pass (UVM_ERROR = 0)
Misaligned Access (ERROR)	HSIZE=010 with HADDR[1:0] != 2'b00	HRESP=1 for two cycles; HREADYOUT=0 then 1

Reset Behavior	Assert HRESETn=0 mid-simulation	All memory cleared; FSM returns to ST_IDLE

5.1 Directed Test Results

All directed test cases in ahb_tb.sv passed successfully. The simulation log showed [PASS] for every read operation, and the final banner reported FINAL RESULT: ALL TESTS PASSED with error_count = 0.

5.2 UVM Randomized Test Results

The UVM simulation executed 10 write-then-read pairs across randomly generated addresses within the constrained range. The scoreboard reported zero UVM_ERROR messages. All SCB_PASS entries confirmed that read data matched the byte-accurate reference model for every transaction.

6. Project File Structure

The project is organized into two layers: the RTL design under test and the verification environment. Both the directed testbench and the UVM environment share the same DUT and interface definition.

RTL / Design Files

- ahb5_subordinate.sv — Production DUT (combinatorial HRDATA path).
- ahb.sv — Earlier registered-output variant for reference and regression.

Verification Infrastructure

- ahb_if.sv — Virtual interface definition shared by all UVM components.
- top.sv — UVM top-level module; instantiates DUT, connects interface, launches test.
- ahb_uvm_pkg.sv — Single-file compilation package for all UVM classes.
- ahb_tb.sv — Self-contained directed testbench (no UVM dependency).

7. Conclusion

This project successfully delivered a fully AHB5-compliant subordinate memory controller alongside a comprehensive two-tier verification environment. The design correctly implements all mandatory AHB5 transfer types, byte-lane strobing, and the two-cycle ERROR response protocol for misaligned accesses.

The verification strategy demonstrated the complementary strengths of both directed and constrained-random methodologies. The directed testbench provided deterministic coverage of specific corner cases, while the UVM environment offered scalable, reusable infrastructure suitable for regression testing across design iterations.

All test scenarios passed with zero errors, providing high confidence in the functional correctness of the ahb5_subordinate design. The parameterized architecture and modular UVM component structure ensure the project can be readily extended to support wider data buses, deeper memories, or additional AHB5 features such as burst transfers and exclusive access support.

8. References

- ARM Limited. AMBA AHB Protocol Specification, Version 5 (AHB5). ARM IHI 0033B.b, 2012.
- Accellera Systems Initiative. Universal Verification Methodology (UVM) 1.2 User Guide. 2015.
- IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE Std 1800-2017.
- Sutherland, S., Davidmann, S., Flake, P. SystemVerilog for Design, 2nd Edition. Springer, 2006.
- Spear, C., Tumbush, G. SystemVerilog for Verification, 3rd Edition. Springer, 2012.

End of report

Document prepared by Omar Ahmed

Senior 1 EECE Student at CU • 2026